operator. In this case, the function must accept a value of type `T` as the sole parameter. Here's where you do the hand-over-hand locking. To start with, you lock the mutex on the `head` node ❽. It's then safe to obtain the pointer to the `next` node (using `get()` because you're not taking ownership of the pointer). If that pointer isn't `NULL` ❾, you lock the mutex on that node ❿ in order to process the data. Once you have the lock on that node, you can release the lock on the previous node ⓫ and call the specified function ⓬. Once the function completes, you can update the `current` pointer to the node you just processed and `move` the ownership of the lock from `next_lk` out to `lk` ⓭. Because `for_each` passes each data item directly to the supplied `Function`, you can use this to update the items if necessary or copy them into another container, or whatever. This is entirely safe if the function is well behaved, because the mutex for the node holding the data item is held across the call.

`find_first_if()` ⓮ is similar to `for_each()`; the crucial difference is that the supplied `Predicate` must return `true` to indicate a match or `false` to indicate no match ⓯. Once you have a match, you just return the found data ⓰ rather than continuing to search. You could do this with `for_each()`, but it would needlessly continue processing the rest of the list even once a match had been found.

`remove_if()` ⓱ is slightly different, because this function has to actually update the list; you can't use `for_each()` for this. If the `Predicate` returns `true` ⓲, you remove the node from the list by updating `current->next` ⓳. Once you've done that, you can release the lock held on the mutex for the `next` node. The node is deleted when the `std::unique_ptr<node>` you moved it into goes out of scope ⓴. In this case, you don't update `current` because you need to check the new `next` node. If the `Predicate` returns `false`, you just want to move on as before ㉑.

So, are there any deadlocks or race conditions with all these mutexes? The answer here is quite definitely *no*, provided that the supplied predicates and functions are well behaved. The iteration is always one way, always starting